

UNIVERSIDAD AUTÓNOMA DE OCCIDENTE

Facultad de Ingeniería – Núcleo MIDIA

Estructuras de Datos y Algoritmos 1

TALLER – ANÁLISIS DE ALGORITMOS (CORTE 2)

Nombre: Sergio Gomez

Código: 2226785

Grupo: 51

Fecha: 29/09/2026

ALGORITMOS NO RECURSIVOS

Se cuenta como una instrucción cada asignación, comparación, incremento, return, llamada o impresión. En un for la inicialización cuenta una vez, la condición se evalúa una vez más que las iteraciones y el incremento se ejecuta una vez por iteración. n es el tamaño del arreglo (`a.length`) y se toma par para que $n/2$ sea entero.

PRIMERA PARTE: cálculo del número total de instrucciones ejecutadas

a) item1

```
item1.js
1  function item1(a){
2      let res ="Si se cumple";
3      let final = a.length-1;
4      for(let i=0; i<a.length/2; i++){
5          if (a[i] != a[final]){
6              res ="no se cumple";
7              break;
8          }
9          final--
10     }
11     return res;
12 }
```

item1 verifica si un arreglo es palíndromo comparando los extremos hacia el centro. El peor caso se da cuando el arreglo sí es palíndromo: el break nunca se ejecuta y el for hace las $n/2$ vueltas, cada una con el if y el final--.

Ecuación de instrucciones:

$$T(n) = 2 + 1 + (n/2 + 1) + n/2 + n/2 + n/2 + 1$$

$$T(n) = 4 + (n/2 + 1) + 3(n/2)$$

$$T(n) = 2n + 5$$

b) item2

```
item2.js

1  function item2(n){
2      let tope = Math.floor(Math.sqrt(n));
3      let res = true;
4
5      for(let i=2; i<=tope; i++){
6          if (n%i==0){
7              res =false;
8              break;
9          }
10     }
11     return res;
12 }
13
14 for (let pos=2; pos<=20; pos++){
15     console.log(pos+' '+item2(pos))
16 }
```

item2 dice si n es primo probando divisores desde 2 hasta $\sqrt{n}$. El peor caso es que n sea primo, porque ningún divisor corta el ciclo y hace tope - 1 vueltas, con $\text{tope} = \text{floor}(\sqrt{n})$.

Ecuación de instrucciones para una llamada:

$$T(n) = 3 + \text{tope} + (\text{tope} - 1) + (\text{tope} - 1) + 1$$

$$T(n) = 4 + \text{tope} + 2(\text{tope} - 1)$$

$$T(n) = 3 \cdot \text{tope} + 2 = 3 \cdot \text{floor}(\sqrt{n}) + 2$$

Para el ciclo de 2 a 20 se calcula cada llamada por separado. Un primo cuesta $3 \cdot \text{tope} + 2$, y un compuesto cuesta $3d + 2$, donde d es su menor divisor (el break corta ahí el ciclo).

Número	Tipo	Tope / Divisor	Instrucciones
2	Primo	tope = 1	5
3	Primo	tope = 1	5
4	Compuesto	d = 2	8
5	Primo	tope = 2	8
6	Compuesto	d = 2	8
7	Primo	tope = 2	8
8	Compuesto	d = 2	8
9	Compuesto	d = 3	11
10	Compuesto	d = 2	8
11	Primo	tope = 3	11
12	Compuesto	d = 2	8
13	Primo	tope = 3	11
14	Compuesto	d = 2	8
15	Compuesto	d = 3	11
16	Compuesto	d = 2	8
17	Primo	tope = 4	14
18	Compuesto	d = 2	8
19	Primo	tope = 4	14
20	Compuesto	d = 2	8

Las 19 llamadas suman 170. El for de pos agrega 1 (inicialización) + 20 (condición) + 19 (incremento) + 19 (llamada, concatenación y console.log) = 59.

Total = 170 + 59 = 229 instrucciones

c) item3

```
item3.js
1  function item3(a){
2      let cont=0;
3      let suma = 0;
4      for (let i=2; i<a.length-2; i++){
5          for(let j=0; j<=a.length/2; j++){
6              if (a[j]%2==0){
7                  cont++;
8              }else{
9                  suma += a[j];
10                 a[j]+=2;
11             }
12         }
13     }
14     return cont;
15 }
```

item3 cuenta los pares de la primera mitad del arreglo y, con los impares, los acumula en suma y les suma 2. El for externo hace $n - 4$ vueltas (i de 2 a $n - 3$) y el interno $n/2 + 1$ (j de 0 a $n/2$). El peor caso es que todos sean impares: el else ejecuta 2 instrucciones y con el if cada vuelta interna cuesta 3.

Cada vuelta del for externo cuesta:

$$1 + (n/2 + 2) + (n/2 + 1) + 3(n/2 + 1) = 5n/2 + 7$$

Ecuación de instrucciones:

$$T(n) = 2 + 1 + (n - 3) + (n - 4) + (n - 4)(5n/2 + 7) + 1$$

$$T(n) = 2n - 3 + (5n^2/2 + 7n - 10n - 28)$$

$$T(n) = 5n^2/2 - n - 31$$

Con $n = 6$ da $2 + 1 + 3 + 2 + 2(22) + 1 = 53$, y la fórmula da $90 - 6 - 31 = 53$.

d) item4 e item5

```
item4.js
1  function item4(a){
2      let veces=0;
3
4      for (let i=0; i<a.length; i++){
5          a[i] += item5(a,i);
6          if (a[i]>2){
7              veces++;
8          }
9      }
10     return veces;
11 }
12
13 function item5(a,b){
14     let cont=0;
15
16     for (let i=0; i<a.length; i++){
17         if (i!=b){
18             if (a[i]>a[b]){
19                 cont++;
20             }
21         }
22     }
23     return cont;
24 }
```

item5(a, b) cuenta cuántos elementos son mayores que a[b]. Su peor caso es que el segundo if sea verdadero en las n - 1 posiciones distintas de b, así que el cont++ se ejecuta n - 1 veces. item4 llama a item5 en cada vuelta y su peor caso es que todos los a[i] queden mayores que 2 para que veces++ se ejecute siempre.

Ecuación de item5:

$$T5(n) = 2 + (n + 1) + n + n + 2(n - 1) + 1$$

$$T5(n) = 5n + 2$$

Cada vuelta de item4 cuesta 1 (la suma) + T5(n) + 1 (if) + 1 (veces++) = 5n + 5. Ecuación de item4:

$$T(n) = 2 + (n + 1) + n + n(5n + 5) + 1$$

$$T(n) = 4 + 2n + 5n^2 + 5n$$

$$T(n) = 5n^2 + 7n + 4$$

SEGUNDA PARTE: cálculo de complejidades

a) Casos de la primera parte

item1 es $O(n)$: tiene un solo for de $n/2$ vueltas y $T(n) = 2n + 5$, cuyo término dominante es lineal.

item2 es $O(\sqrt{n})$ por llamada: el for llega como máximo a $\sqrt{n}$ y $T(n) = 3 \cdot \sqrt{n} + 2$. El ciclo de pos de 2 a 20 es $O(1)$ porque el rango es fijo; si fuera hasta N sería $O(N \cdot \sqrt{N})$.

item3 es $O(n^2)$: dos for anidados que dependen de n , y en $T(n) = 5n^2/2 - n - 31$ domina el término n^2 .

item4 es $O(n^2)$: recorre el arreglo n veces y en cada vuelta llama a item5, que es $O(n)$, así que queda $n \cdot n$. Coincide con $T(n) = 5n^2 + 7n + 4$.

b) BubbleSort, printArray y main (GeeksforGeeks)

bubbleSort.java

```
1 static void bubbleSort(int arr[], int n){
2     int i, j, temp;
3     boolean swapped;
4     for (i = 0; i < n - 1; i++) {
5         swapped = false;
6         for (j = 0; j < n - i - 1; j++) {
7             if (arr[j] > arr[j + 1]) {
8                 temp = arr[j];
9                 arr[j] = arr[j + 1];
10                arr[j + 1] = temp;
11                swapped = true;
12            }
13        }
14        if (swapped == false)
15            break;
16    }
17 }
```

printArray.java

```
1 static void printArray(int arr[], int size){
2     int i;
3     for (i = 0; i < size; i++)
4         System.out.print(arr[i] + " ");
5     System.out.println();
6 }
```

main.java

```
1 public static void main(String args[]){
2     int arr[] = { 64, 34, 25, 12, 22, 11, 90 };
3     int n = arr.length;
4     bubbleSort(arr, n);
5     System.out.println("Sorted array: ");
6     printArray(arr, n);
7 }
```

bubbleSort es $O(n^2)$. Sus dos for anidados en el peor caso (arreglo en orden inverso) hacen $(n - 1) + (n - 2) + \dots + 1 = n(n - 1)/2$ comparaciones. En el mejor caso, con el arreglo ya ordenado, swapped queda en false en la primera pasada y termina en $O(n)$.

printArray es $O(n)$ porque un solo for recorre los n elementos.

main es $O(n^2)$: solo llama a bubbleSort, que es $O(n^2)$, y a printArray, que es $O(n)$, y domina el n^2 . El arreglo del ejemplo tiene 7 elementos fijos, pero en función de n la complejidad es cuadrática.

c) item6

item6.js

```
1 function item6(a){
2     for (let i=1; i<a.length-1; i++){
3         for (let j=0; j<Math.trunc(Math.log2(a.length)); j++){
4             console.log(a[i]+' '+a[j]);
5         }
6     }
7 }
```

item6 es $O(n \log n)$: el for externo hace $n - 2$ vueltas y el interno hace $\text{trunc}(\log_2(n))$ vueltas sin importar i , así que el total es aproximadamente $(n - 2) \cdot \log_2(n)$.

d) opera e item7

```
opera.js

1  function opera(b,c){
2      let veces = 0;
3      for (let x=b; x<=c; x++ )
4      {
5          if (item2(x))
6          {
7              veces++;
8          }
9      }
10     return veces;
11 }
12
13 function item7(a){
14     for (let i=0; i<a.length-1; i++){
15         a[i] = opera( i , a.length-1)
16     }
17 }
```

`opera(b, c)` llama a `item2` en cada valor de `b` a `c`, y cada llamada es $O(\sqrt{x})$, así que `opera` es $O((c - b + 1) \cdot \sqrt{c})$. `item7` llama a `opera(i, n - 1)` para i de 0 a $n - 2$, que son $n - 1$ llamadas a `item2` de costo $O(\sqrt{n})$. La suma da aproximadamente $(n^2/2) \cdot \sqrt{n} = n^{5/2}/2$, por lo que `item7` es $O(n^{5/2})$.